

Vision Transformers (ViT)

Métodos Generativos, curso 2026-2027

Guillermo Iglesias, guillermo.iglesias@upm.es

Edgar Talavera Muñoz, e.talavera@upm.es

29 de septiembre de 2026

Escuela Técnica Superior de Ingeniería de Sistemas Informáticos | UPM



Los Vision Transformers (ViTs) son una arquitectura presentada en el artículo *"An image is worth 16x16 words: Transformers for image recognition at scale"* del año 2020 [1] dentro del área de la **visión por computador**.

- Utilizan los mecanismos de **atención** para procesar imágenes como **secuencias de píxeles**
- Utilizan únicamente el **encoder** para generar **descriptores** de imágenes

Ventajas

- Captura **dependencias globales** de la imagen, es decir, correlaciona elementos muy distantes
- Mejora los resultados a la hora de interpretar o generar imágenes
- Simplifican el **condicionamiento** de las arquitecturas

Desventajas

- Necesidad de **grandes datasets**¹
 - Muchas veces se contrarresta con un **pretraining**, pero no siempre es posible, y sigue siendo muy costoso
 - Hay investigación que intenta reducir el coste [2]

¹Las CNNs saben que los píxeles cercanos están correlacionados (inductive bias [1]), estos modelos aprenden la geometría desde cero.

Los principales pasos en un ViT son los siguientes:

1. Tokenización de la imagen
2. Procesamiento de tokens
3. Encoding de los tokens
4. Procesamiento con Neural Network (NN)
5. Generación de la salida

Tokenización de la imagen

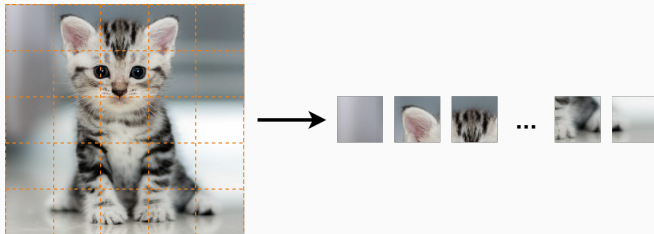
De la misma manera que los Transformers en **Natural Language Processing (NLP)** necesitan que el texto sea procesado para ser entendible por un modelo de Artificial Intelligence (AI), las imágenes han de ser **preprocesadas** para ser tratadas.



→ [1.56, 0.45, -0.23, ..., 3.2]

Tokenización de la imagen: Generación de parches

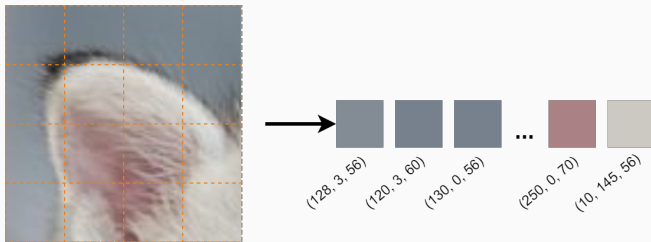
Los ViT procesan una imagen como una **secuencia de tokens**. Cada token es un **parche** de la imagen total, de tamaño $P_H \times P_W \times C$ (alto, ancho, canales).



*El tamaño más estándar para los parches es 16x16 píxeles

Tokenización de la imagen: Flatten de parches

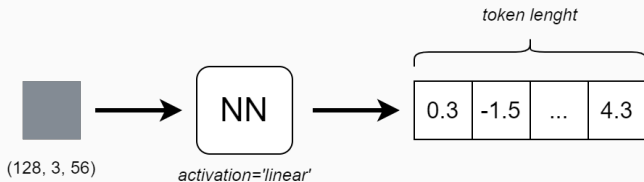
Cada parche es **vectorizado** aplicando un flatten.



Cada posición representa el valor de color del píxel.

Tokenización de la imagen: Proyección lineal de los tokens

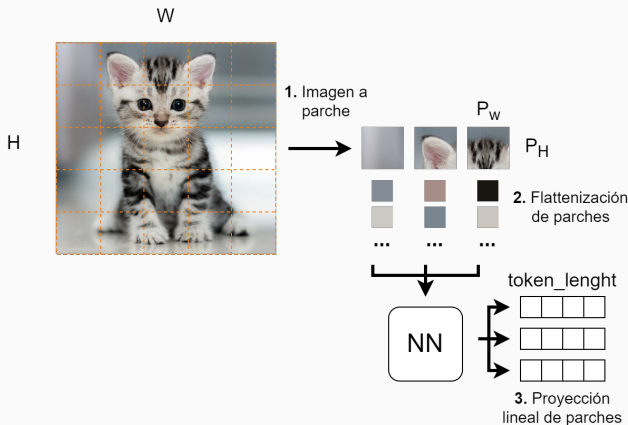
Una práctica común es proyectar los tokens a un espacio latente de un tamaño arbitrario. Se utiliza para mejorar la **representación de los parches**, lo que es equivalente a los **embeddings de palabras** en NLP.



*En [1] el *token lenght* se define a 768.

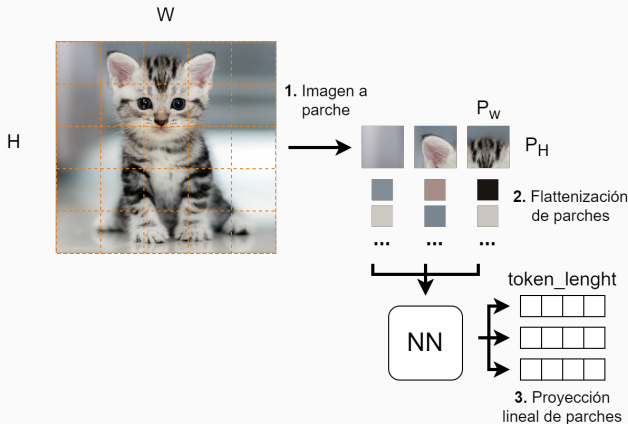
Tokenización de la imagen: Resumen

Con todo el proceso definido anteriormente, es posible generar una **representación latente** de una imagen, dividida en parches, que además pueden adaptarse al proceso de **aprendizaje**.



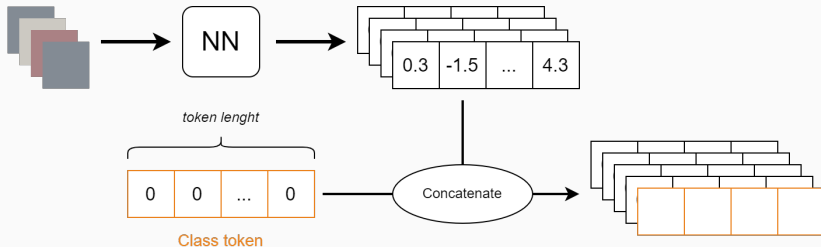
Procesamiento de tokens

El siguiente paso consiste en procesar los tokens para añadir **información extra** necesaria para mantener las relaciones espaciales y generar una salida.



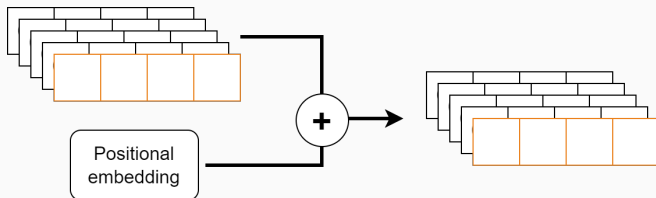
Procesamiento de tokens: Concatenar el *class token*

El *class token* es un vector que siempre se inicializa a 0, usado para generar la salida de *predicción tras en encoding*.



Procesamiento de tokens: Position embedding

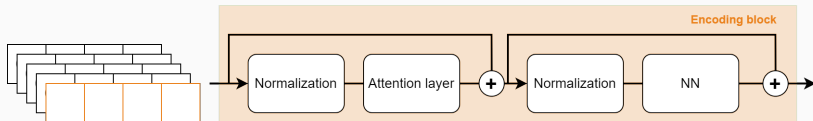
Para mantener las **relaciones espaciales** de los parches de una imagen se añade un **positional embedding**² que **ordena** los embeddings. Esto ya se hacía en los **Transformers** para NLP.



²Recalamos que el positional embedding es **sumado** a los anteriores. Normalmente se usa una función **sinusoidal** o la salida de un perceptrón.

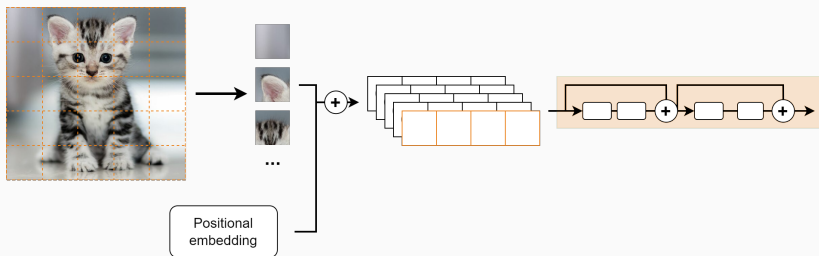
Encoding

La información generada previamente se procesa con bloques de encoding usando capas de atención. Se sigue el mismo procedimiento que en los Transformers vanilla.



Encoding: Intuición

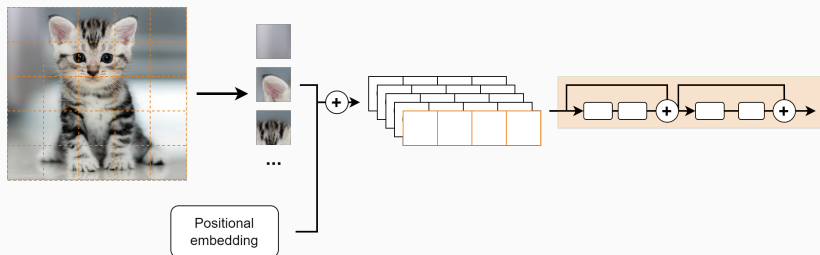
En este punto cabe recordar que el **encoder** está procesando cada parche de la imagen, **proyectado** a un espacio latente que guarda las **relaciones entre parches**. De esta forma es posible **extraer características** de las imágenes.



Salida del modelo

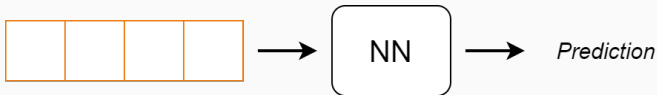
Una vez procesada la información, se genera la **salida** de la red de neuronas. Hasta este punto, la arquitectura del ViT es general y puede usarse para múltiples problemas. Pero en este ejemplo se explicará cómo hacer una **clasificación**.

Para ello se usa el **class token** anteriormente definido.



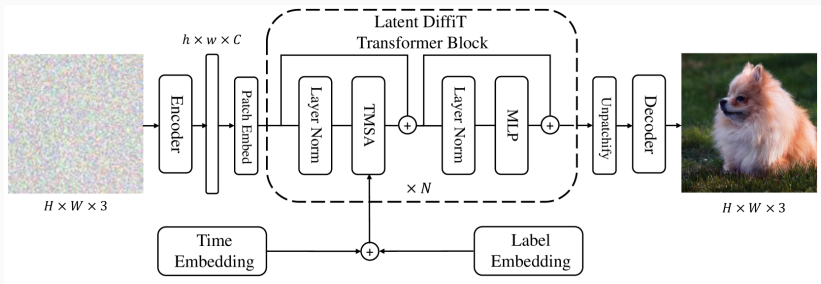
El class token es finalmente procesado por una **cabeza de clasificación** compuesta por un **perceptrón**.

Normalmente, este perceptrón no es más que una capa con activación lineal.



Generación de contenido: DiffiT

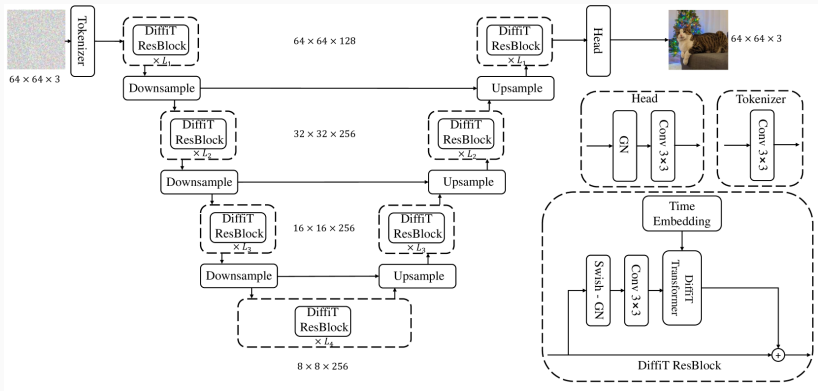
Existen arquitecturas como **Diffusion Vision Transformer (DiffiT)** [3] que combinan la potencia de los ViT con la capacidad de generación de los **Diffusion models** para generar contenido.



[3]

Generación de contenido: DiffiT

Combinando una arquitectura de **Autoencoder (AE)** e introduciendo bloques de ViT con un **embedding temporal** consiguen generar los mejores resultados del estado del arte³.



[3]

³Por fin, llegamos a lo mejor de lo mejor.

De ViT a Diffusion Transformers (DiT / DiffiT)

Para que un ViT genere imágenes, cambiamos el paradigma:

- **Entrada:** Ya no recibe una imagen limpia, sino una **imagen ruidosa** x_t en un instante de tiempo t .
- **Procesamiento:** Se divide x_t en parches exactos como en el ViT clásico.
- **Salida:** En lugar de predecir una clase con un *prediction token*, el modelo procesa todos los tokens para predecir el **tensor de ruido** $\epsilon_\theta(x_t, t)$ del mismo tamaño que la imagen original.

Condicionamiento con tiempo y clase

En un modelo de difusión, el modelo **debe saber en qué paso t se encuentra** y qué está generando (condición c , como texto o clase)⁴.

DiffT inyecta esta información usando **Adaptive Layer Normalization (adaLN)**[4]⁵:

- Se extraen *embeddings* del tiempo t y la condición c .
- A partir de estos, un Multilayer Perceptron (MLP) calcula parámetros de escala (γ) y desplazamiento (β).
- Antes de cada bloque de atención, los tokens se modulan:

$$\text{adaLN}(h, t, c) = \gamma(t, c) \cdot \text{LayerNorm}(h) + \beta(t, c)$$

⁴Esto es opcional, por supuesto, pero la mayoría de modelos lo usan.

⁵Si observais la cita, veréis que esto se propuso para GANs, pero se usa en todo tipo de modelos.

Bloques DiffT: Atención Jerárquica

El cálculo de la **Self-Attention** tiene una complejidad de $\mathcal{O}(N^2)$. Para latentes muy grandes, esto desborda la memoria de la Graphics Processing Unit (GPU).

Para solventarlo, DiffT propone un mecanismo inspirado en arquitecturas jerárquicas como los *Swin Transformers* (Shifted Window Transformer):

- **Atención Local (*Window Attention*):** Los tokens se agrupan en ventanas espaciales pequeñas. La atención se calcula solo entre los tokens de la misma ventana (capturando texturas locales).
- **Conexiones Shifted:** Entre cada capa las ventanas se desplazan para permitir el flujo de información.
- **Atención Global reducida:** En las capas más profundas, se reduce la resolución espacial fusionando parches (*Patch Merging*), permitiendo una atención global sobre la estructura principal de la imagen.

Des-tokenización y loss

Tras atravesar los bloques de codificación obtenemos una **secuencia final de tokens** z_L .

La salida generativa requiere revertir la tokenización:

1. Se aplica una capa lineal final a cada token.
2. Los N tokens se reordenan espacialmente (*un-patchify*) para reconstruir la imagen original de $H \times W \times C$.

Este tensor final es el **ruido predicho** $\epsilon_{\theta}(x_t, t)$.

La red se optimiza minimizando el **Mean Squared Error (MSE)** frente al ruido real ϵ que se inyectó en el paso *forward*:

$$\mathcal{L}_{MSE} = \|\epsilon - \epsilon_{\theta}(x_t, t, c)\|^2$$

- [1] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al.
An image is worth 16x16 words: Transformers for image recognition at scale.
arXiv preprint arXiv:2010.11929, 2020.
- [2] Li Yuan, Yunpeng Chen, Tao Wang, Weihao Yu, Yujun Shi, Zi-Hang Jiang, Francis EH Tay, Jiashi Feng, and Shuicheng Yan.
Tokens-to-token vit: Training vision transformers from scratch on imagenet.
In Proceedings of the IEEE/CVF international conference on computer vision, pages 558–567, 2021.

- [3] Ali Hatamizadeh, Jiaming Song, Guilin Liu, Jan Kautz, and Arash Vahdat.

Diffit: Diffusion vision transformers for image generation.

In European Conference on Computer Vision, pages 37–55.

Springer, 2024.

- [4] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila.

Analyzing and improving the image quality of stylegan.

In 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 8107–8116. IEEE, 2020.

Cita APA: Iglesias, G. and Talavera, E. (2026, septiembre 29). *Vision Transformers (ViT)* [Diapositivas de la asignatura Métodos Generativos]. GitHub. https://guillermoih.github.io/M-todos-Generativos/Vision_Transformers.pdf

Tema de las diapositivas "Metropolis" lincenciado bajo LPPL 1.3c

BibTeX

```
@slides{Vision_Transformers_MG_2026,  
  author = {Iglesias, Guillermo and Talavera, Edgar},  
  title = {Vision Transformers (ViT) [Diapositivas de la asignatura Métodos Generativos]},  
  year = {2026},  
  month = {9},  
  howpublished={\url{https://guillermoih.github.io/M-todos-Generativos/Vision_Transformers.pdf}},  
  note = {{Online; accessed 29 de septiembre de 2026}},  
  organization={ETSISI. Universidad Politécnica de Madrid}  
}
```